

CRUD WITH IMAGE UPLOADING

WORKING:

- **HOW VALUE ADDED IN VIEW GET STORED IN DATABASE.**
- **HOW CONTROLLER CONNECTS VIEW AND MODEL.**
- **HOW MODEL CONNECTS PROJECT WITH DATABASE.**
- **HOW VALUES STORED IN DATABASE CAN BE FETCHED AND DISPLAYED IN VIEW.**

IMPORTANT NOTE:

- Image is not directly stored as file in database because it can cause the memory shortage
So we only store the path of the image in string in database and store the image file in a specific folder in the Project.
- To do this we need to follow following steps:

Steps to follow in Class:

REQUIREMENTS:

1. We require to get Image File.
2. Once we get the file, we require to store it in Project.
3. Store its path in Database.
4. Ensuring the image file will not be saved in Database.

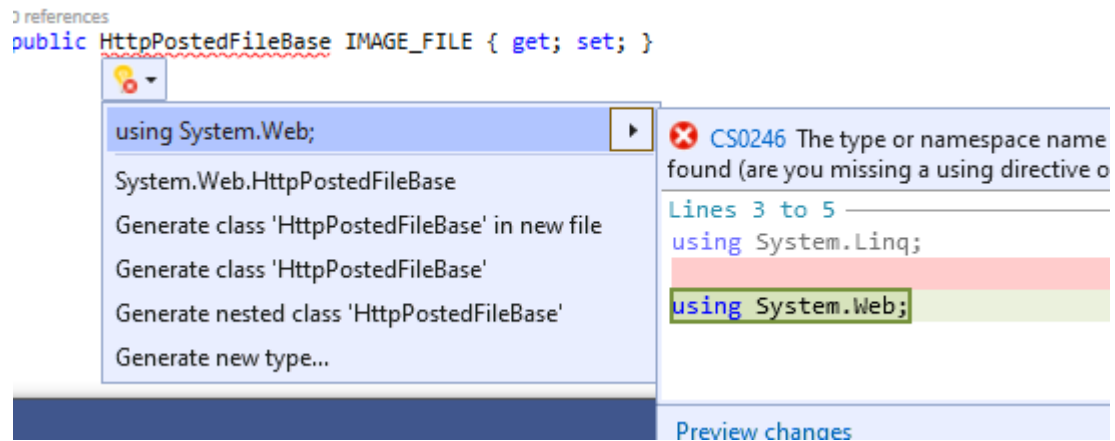
Suppose this is the class in which we require to get Image :

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace HospitalMS.Controllers
{
    0 references
    public class Product
    {
        0 references
        public int ID { get; set; }
        0 references
        public string PRODUCT_NAME { get; set; }
        0 references
        public string PRODUCT_IMAGE { get; set; }
        0 references
        public decimal Price { get; set; }
    }
}
```

Step 1:

- Add another property with name IMAGE_FILE in which we get image file.
- Add an extension “[HttpPostedFileBase](#)” in place of datatype and add its reference by right clicking in the error and clicking “Show Potential Fixes” .



- The reference link will be added:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace HospitalMS.Controllers
{
    0 references
    public class Product
```

Step 2:

```
0 references
public HttpPostedFileBase IMAGE_FILE { get; set; }
```

- We don't require to create a column for this property in database because we don't want to store this file in the db.

ERROR:

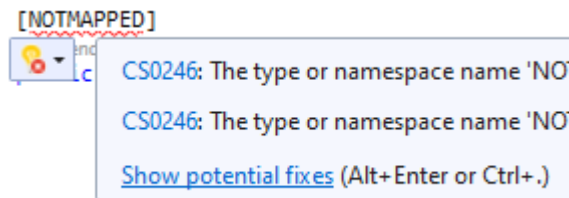
- If we run the application as it is it will show error because it will not find the Column in db to store its value.

SOLUTION:

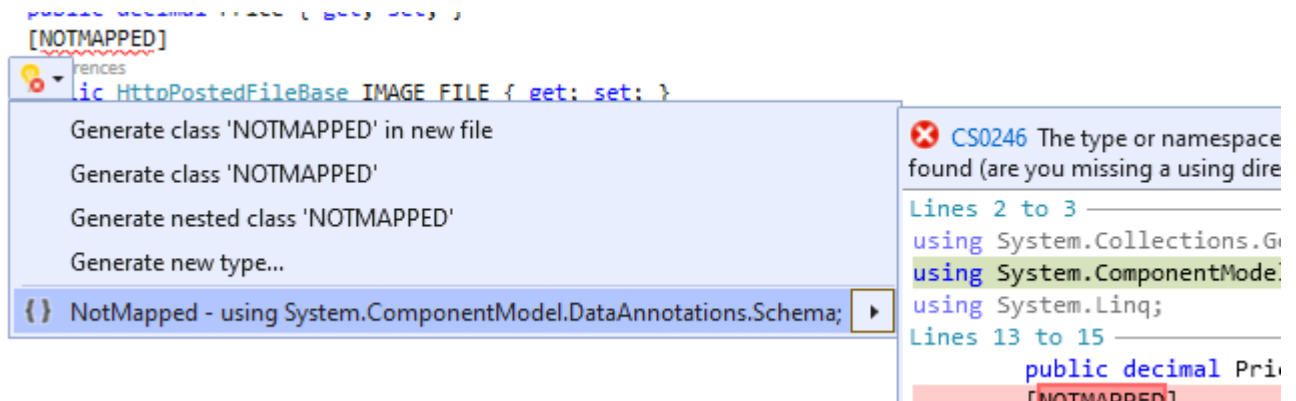
- We will an attribute “NOT MAPPED” so the image file property could be ignored and will not be stored in db.

```
[NOTMAPPED]
0 references
public HttpPostedFileBase IMAGE_FILE { get; set; }
```

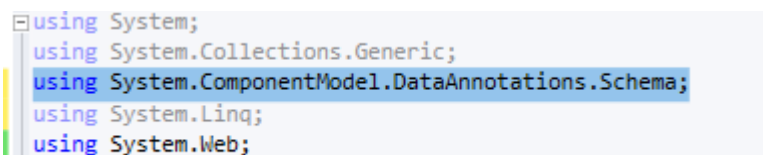
- To resolve this error add the reference library by following these steps:
Right Click error > Click Show Potential Fixes



Choose the Library as shown in picture.



The reference will be added



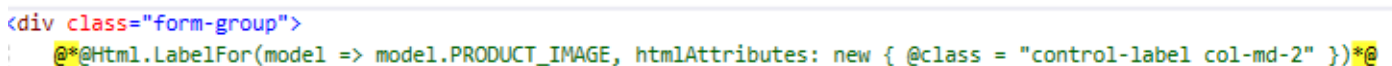
Steps to Follow in View:

REQUIREMENTS:

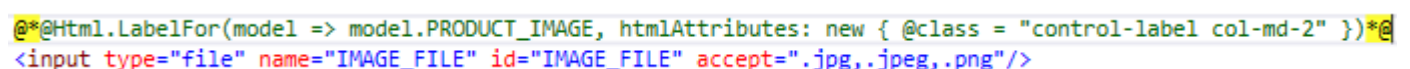
1. When running the product, we want to replace the request for entering a file path with a "Choose File" option, allowing users to browse and select the image directly
2. The selected file should be saved in a specific folder.
3. When Return to Index, image uploaded should be displayed as Image not as path in string.

Step1: Working in Create View

- Go to Create View , Comit the code for getting image.



- Add input tag:



- Set the input type as "file." Use the same name as given in the class (it should match). give id (can be of use). Use the "accept" attribute to allow only specific image file types (add a "." before each extension). Common accepted formats are .png, .jpeg, and .jpg.
- Edit `Html.BeginForm` in Create View. Provide the action path by providing the method name and controller name inside parentheses (). Set the form's method to `POST`. To allow the form to upload file values, include `enctype="multipart/form-data"`.
- ```
using (Html.BeginForm("Create", "Products", FormMethod.Post, new { enctype="multipart/form-data"}))
```
- Create is the Method name, Products is the Controller name (path where front end request will go).

## Step 2: Working on Index View:

- To display the image itself instead of its file path as text, go to the Index View page. Inside the foreach loop, update the code to show the image by rendering it using an `<img>` tag.

```
<td>
 @* @Html.DisplayFor(modelItem => item.PRODUCT_IMAGE)*@
</td>
```

- Use the `<img>` tag inside the foreach loop to display the image dynamically. Use the HTML helper `@Url.Content(item.PRODUCT_IMAGE)` to set the image path, as the path will change based on the uploaded image. Set the height and width of the image using the height and width attributes.

```
<td>
 @* @Html.DisplayFor(modelItem => item.PRODUCT_IMAGE)*@

</td>
```

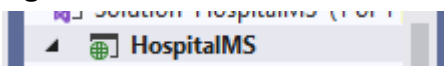
## Steps to Follow in Controller:

### REQUIREMENTS:

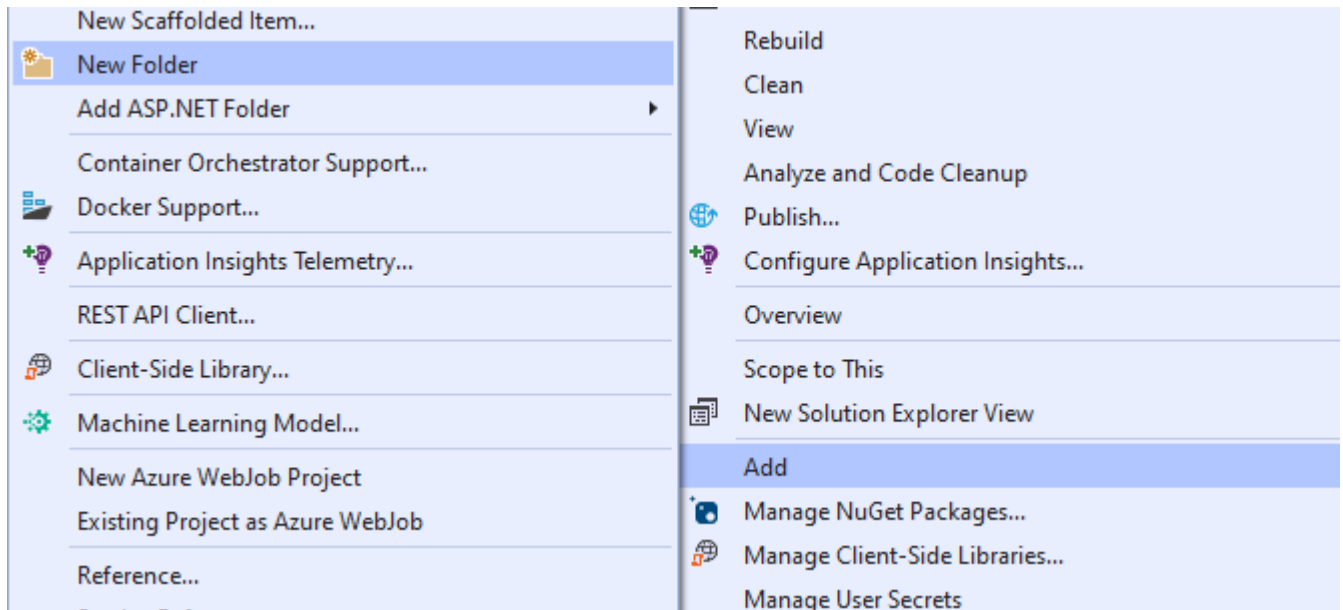
1. Getting both image file and name from the frontend.
2. Making the Image filename unique to prevent replacement of files of same names.
3. Providing the Path of the folder to store Image in Project.
4. Combining the Path and file name and storing it in the Database.

## Step 1: Add Folder:

- First Add the folder in which we require to store the images in the project.
- Right Click Solution



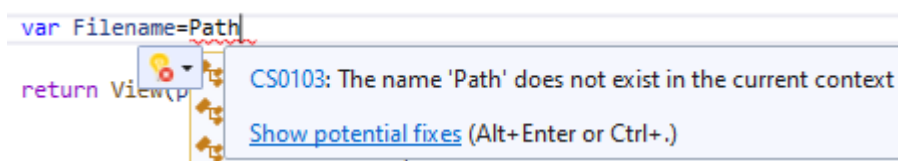
- Click Add> Add new Folder> Give Name (Images).



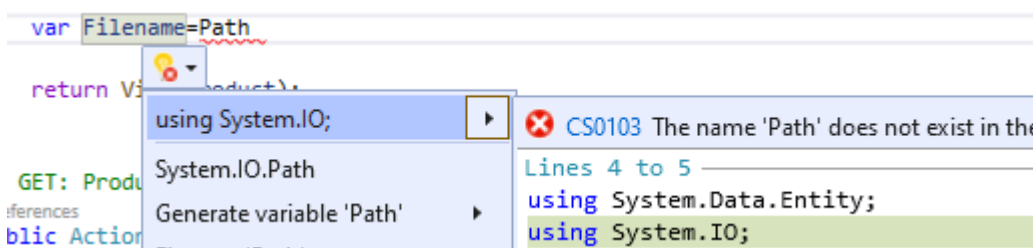
## Step 2: Working on Post Create Method in Controller

### To get File Name:

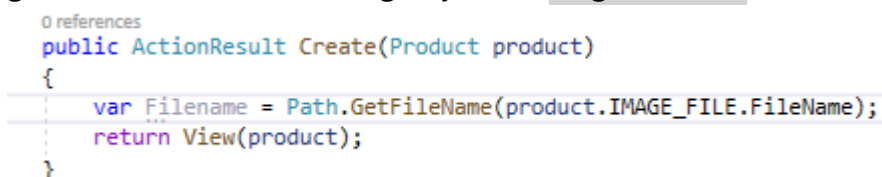
- Declare a variable named "Filename" (could be any name) with dynamic datatype "var".
- Write Path in this variable and then add its built-in reference Library. Drag arrow to error and click show potential fixes from the suggestions.



- Then add Library :



- Use the built-in method `GetFileName` to get the file name from the product object. Write code like this: "In the product object, there is an attribute called `IMAGE_FILE`. Use this attribute to get the file name." This will give you the `Original Name` of the file



### To Create Unique File Name:

- Declare a variable `_Filename` with the dynamic datatype `var` to store the unique file name.
- Add the current time (`DateTime.Now`) to the file name and convert it to a string using the `ToString()` method.
- Use the format `ToString("hhmmssff")` to take time from hours to milliseconds for uniqueness.

```
var _Filename = DateTime.Now.ToString("hhmmssff");
```

- Add the Original File name.(`Filename`).

```
var Filename = Path.GetFileName(product.IMAGE_FILE.FileName);
var _Filename = DateTime.Now.ToString("hhmmssff")+ Filename;
```

- Now the Name of file has become unique so even files with the same will get the unique name before storing in the folder so no image will be replaced due to existing name.

### To Provide Path with Unique Name:

- Declare a variable “`Filepath`” with the dynamic datatype “`var`” to store the unique file Path.
- Use a built in method in Path “`Combine`” which will combine the Path of image in “`Images`” Folder in existing project with the unique name of the file.

```
var _Filename = DateTime.Now.ToString("hhmmssff")+ Filename;
var Filepath = Path.Combine(Server.MapPath("~/Images/") + _Filename);
```

- Server can be local or online, in this case the server is local.
- “`MapPath`” means provide the path of the existing Project. It's important to provide the path of the Project that Exist.
- To provide path write “`~/Foldername/`”. In this project the Folder name is “`Images`” where we require to store the Image files.
- Add the unique name “`_Filename`”.
- This full path will be used to **Store the image path** on your local server.

### To Save Image Path in Database:

- To store the name of the image in the database:

```
product.PRODUCT_IMAGE = "~/Images/" + _Filename;
```

- Put the image's file name ( "`_Filename`") together with the folder "`~/Images/`").
- Save the result in `product.PRODUCT_IMAGE`.

```
db.Products.Add(product);
```

- Use `db`, which is an object of the `DbContext` class, to work with the database.
- The `Products` table in the database is used to store values.
- The `Add` function (a built-in function) is used to add new values to the table.
- The `product` object (an object of the `Product` class) holds the image's full path (folder + unique name).

- Adding product to the Products table saves the image's full path in the database.

### To Save Changes:

### PROBLEMS IN GETTING FILE FROM USER:

- Image file size can be very large (e.g. 500MB)
- Making sure image is provided.

### Steps to Solve the Problems:

```
if(product.IMAGE_FILE.ContentLength < 3000000)
{
 if(db.SaveChanges()>0)
 {
 product.IMAGE_FILE.SaveAs(Filepath);
 }
 return RedirectToAction("Index");
}
else
{
 ViewBag.msg = "Filesize exceeds 3MB, Try Again";
}
return View(product);
```

- To make Sure the uploaded file size (`product.IMAGE_FILE.ContentLength<3000000`) is less than 3MB.
- If the file size is less than 3MB, it tries to save changes to the database using `db.SaveChanges()`.
- If `db.SaveChanges()` is successful (returns a value greater than 0), the file is saved to the location (`Filepath`) using `product.IMAGE_FILE.SaveAs(Filepath)`.
- After saving, the user is redirected to the "Index" page with `return RedirectToAction("Index");`.
- If the file size is more than 3MB, a message "Filesize exceeds 3MB, Try Again" is stored in `ViewBag.msg`.
- Finally, the page is refreshed and the `product` object is returned to the view.

### WORKING ON EDIT:

### REQUIREMENTS:

1. When Editing, image should be displayed rather than its Path.
2. When saved new image should be displayed in Index View.
3. Old image should be deleted after new Image is saved.

### Working on Controller:

#### Step 1: Working on Get Edit Method

- `if (product != null):`

This checks if the variable product is not null (i.e., it contains data).

- `Session["imgPath"] = product.PRODUCT_IMAGE;;`
- If product is not null, this line saves the product image path to the session.
- `Session["imgPath"]`: This is where the image path will be stored. "imgPath" is the Name, and product.PRODUCT\_IMAGE is the value.
- `"imgPath"` is a Name, which we use to access the stored data in the session anywhere in the Project.
- `product.PRODUCT_IMAGE` is the value being stored. It contains the path (URL or file path) of the product image that belongs to the product object.

## Step 2: Working on Post Edit Method

- Copy the code from the post create method as shown and Paste in The post Method of Edit.

```
var Filename = Path.GetFileName(product.IMAGE_FILE.FileName);
var _Filename = DateTime.Now.ToString("hhmmssff") + Filename;
var Filepath = Path.Combine(Server.MapPath("~/Images/"), _Filename);
product.PRODUCT_IMAGE = "~/Images/" + Filename;

if (product.IMAGE_FILE.ContentLength < 3000000)
{
 db.Entry(product).State = EntityState.Modified;
 var oldimgpath = Request.MapPath(Session["imgPath"].ToString());
 if (db.SaveChanges() > 0)
 {
 product.IMAGE_FILE.SaveAs(Filepath);
 if (System.IO.File.Exists(oldimgpath))
 {
 System.IO.File.Delete(oldimgpath);
 }

 return RedirectToAction("Index");
 }
}
else
{
 ViewBag.msg = "Filesize exceeds 3MB, Try Again";
}

return View(product);
```

- `product.IMAGE_FILE.ContentLength<3000000`: The if condition checks if the file size is less than 3 MB (3,000,000 bytes).
- If the file is larger than 3 MB, it jumps to the else block where an error message is shown.
- `db.Entry(product).State = EntityState.Modified;` This tells the Entity Framework that you want to **update** the existing product object in the database and new Product should not be Added.
- `EntityState.Modified;` means this entity has been changed and should be saved back to the database.
- `Session["imgPath"]`: provides the path (location) of the image that was saved in get method edit in the session.
- `var oldimgpath = Request.MapPath(Session["imgPath"].ToString());`
- This line converts a stored image path from the session into a full file path on the server using Request.MapPath. .



- `ToString()` is used to convert into text/ string value.
- Storing the above path in variable `"oldimgpath"`
- `db.SaveChanges() > 0`: Checks if the database was updated successfully.
- `product.IMAGE_FILE.SaveAs(Filepath)`: Saves the new image to a specified location.
- `System.IO.File.Exists(oldimgpath)`: Verifies if the old image exists on the server.
- `System.IO.File.Delete(oldimgpath)`: Deletes the old image file if it exists.
- `RedirectToAction("Index")`: Redirects the Index page.
- If above conditions are false else body will be displayed with message **"Filesize exceeds 3MB, Try Again"**.
- If New image is failed to be saved then will return the product object (data) to the view.

## NULL EXCEPTION ERROR SOLUTION:

### In Class:

- Check [NOTMAPPED] and HttpPostedFileBase.

```
[NotMapped]
6 references
public HttpPostedFileBase IMAGE_FILE { get; set; }
```

### In View:

- Make sure to edit `Html.BeginForm` of the View, { enctype= "multipart/form-data".

```
@using (Html.BeginForm("Create", "Products", FormMethod.Post, new { enctype = "multipart/form-data" }))
```

- Make sure to choose type file.

```
<input class="col-md-3" type="file" name="IMAGE_FILE" id="IMAGE_FILE" accept=".jpg,.jpeg,.png" />
```

- If image is not displayed in index check:

```

```

- If image is not displayed in edit check :

```

```

### In Controller:

- Match the your logic with this code for create method:

```

0 references
public ActionResult Create(Product product)
{
 var Filename = Path.GetFileName(product.IMAGE_FILE.FileName);
 var _Filename = DateTime.Now.ToString("hhmmssff") + Filename;
 var Filepath = Path.Combine(Server.MapPath("~/Images/"), _Filename);
 product.PRODUCT_IMAGE = "~/Images/" + _Filename;
 db.Products.Add(product);

 if (product.IMAGE_FILE.ContentLength < 3000000)
 {
 if (db.SaveChanges() > 0)
 {
 product.IMAGE_FILE.SaveAs(Filepath);
 }
 return RedirectToAction("Index");
 }
 else
 {
 ViewBag.msg = "Filesize exceeds 3MB, Try Again";
 }
 return View(product);
}

```

- Match the your logic with this code for Edit Method.:

```

0 references
public ActionResult Edit(Product product)
{
 var Filename = Path.GetFileName(product.IMAGE_FILE.FileName);
 var _Filename = DateTime.Now.ToString("hhmmssff") + Filename;
 var Filepath = Path.Combine(Server.MapPath("~/Images/"), _Filename);
 product.PRODUCT_IMAGE = "~/Images/" + _Filename;

 if (product.IMAGE_FILE.ContentLength < 3000000)
 {
 db.Entry(product).State = EntityState.Modified;
 var oldimgpath = Request.MapPath(Session["imgPath"].ToString());
 if (db.SaveChanges() > 0)
 {
 product.IMAGE_FILE.SaveAs(Filepath);
 if (System.IO.File.Exists(oldimgpath))
 {
 System.IO.File.Delete(oldimgpath);
 }

 return RedirectToAction("Index");
 }
 }
 else
 {
 ViewBag.msg = "Filesize exceeds 3MB, Try Again";
 }
 return View(product);
}

```

